

GenServer — A Readable Companion to the Elixir Docs

Chapter 4 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why GenServer exists

A GenServer is a process that owns some state and serves requests about it, one at a time. It is Elixir's wrapper around Erlang's `gen_server` (Erlang book, chapter 5), and everything there still applies: calls are monitored requests with a timeout, callbacks run in sequence, and slow work in a callback blocks every caller.

The Elixir docs add one rule that matters more than any API detail: “A GenServer, or a process in general, must be used to model runtime characteristics of your system. A GenServer must never be used for code organization purposes.” Use one when you need state that outlives a call, concurrency, or a unit of failure. For anything else, write functions.

2. The building blocks

One module, two processes

By convention, a `GenServer` module contains both halves:

Part	Functions	Runs in
Client API	<code>start_link/1</code> , <code>get/1</code> , <code>put/2</code> ... wrapping <code>GenServer.call/cast</code>	The caller's process
Server callbacks	<code>init/1</code> , <code>handle_call/3</code> , <code>handle_cast/2</code> , <code>handle_info/2</code> , <code>handle_continue/2</code> , <code>terminate/2</code>	The GenServer process

The module reads as one piece of code but runs in two places. Every mistake in section 4 comes from losing track of which process a line of code runs in.

Callbacks and returns

Callback	Triggered by	Common returns
<code>init/1</code>	<code>start_link</code>	<code>{:ok, state}</code> , <code>{:ok, state, {:continue, term}}</code>
<code>handle_call/3</code>	<code>GenServer.call/3</code> (caller waits, default 5000 ms)	<code>{:reply, reply, state}</code> , <code>{:noreply, state}</code> then <code>GenServer.reply/2</code> later

Callback	Triggered by	Common returns
<code>handle_cast/2</code>	<code>GenServer.cast/2</code> (caller does not wait)	<code>{:noreply, state}</code>
<code>handle_info/2</code>	Any other message: timers, <code>:DOWN</code> , task replies	<code>{:noreply, state}</code>
<code>handle_continue/2</code>	A <code>{:continue, term}</code> return	<code>{:noreply, state}</code>

Naming

name: value	Registered where
<code>MyServer</code> (an atom)	Locally, one per node
<code>{:global, term}</code>	Across a cluster via <code>:global</code> (Erlang book, chapter 6)
<code>{:via, Registry, {MyRegistry, key}}</code>	In a <code>Registry</code> : many named instances (chapter 5)

3. How it actually works

Defaults from use GenServer

`use GenServer` defines a `child_spec/1` (restart `:permanent`, shutdown 5000 ms), and default implementations of the callbacks you don't write. The default `handle_info/2` logs unexpected messages and carries on. **Once you define your own `handle_info/2`, the default is gone.** Any message your clauses don't match raises `FunctionClauseError` and crashes the server.

A call from inside the server

`GenServer.call/3` from the server to itself cannot work: the server would wait for itself. The runtime detects this and exits with `:calling_self`. The easy way to trigger it is to call the module's own client API from a callback.

Late replies

If a call times out, the caller exits (or, if it catches the exit, carries on). On OTP 24 and later the reply is sent to a process alias and discarded if it arrives late. The Elixir docs still warn that a late reply "may arrive at any time later into the caller's message queue," which is true on older OTP versions.

`handle_continue` runs before the next message

Returning `{:continue, term}` from `init/1` or any callback runs `handle_continue/2` immediately after, before the server handles anything else. That lets `init/1` return quickly, so the supervisor can move on, while expensive setup still happens before the first request.

4. Plausible but wrong

4.1 Process calls scattered through the codebase

```
# in a controller
GenServer.call(MyApp.Cart, {:add, user_id, item})
# in a background job
GenServer.cast(MyApp.Cart, {:clear, user_id})
```

Every caller knows the server’s name and its private message format. Renaming a message or moving to one server per user means editing every call site. The process anti-patterns guide calls this *scattered process interfaces*. Fix: expose client functions (`Cart.add(user_id, item)`) in the server module and keep message shapes private to it.

4.2 Expensive init

```
def init(opts) do
  rates = ExchangeAPI.fetch_all!()      # 3 seconds, may fail
  {:ok, %{rates: rates, opts: opts}}
end
```

The supervisor waits for `init/1`, so application boot stalls. If the API is down, the child fails to start, and the application with it. Fix: `{:ok, %{opts: opts}, {:continue, :load}}` and fetch in `handle_continue/2`. If the data can be missing, model a “not loaded yet” state and retry with a timer.

4.3 A hard-coded name

```
def start_link(opts), do: GenServer.start_link(__MODULE__, opts, name: __MODULE__)
```

This allows exactly one instance per node. Tests that start it in parallel collide with `{:error, {:already_started, pid}}`, and a library using this pattern can’t be used twice in one application. Fix: `name: Keyword.get(opts, :name, __MODULE__)`, and have client functions take the server as an argument (defaulting to the module name).

4.4 A `handle_info` that replaces the default

```
def handle_info(:tick, n) do
  Process.send_after(self(), :tick, 1000)
  {:noreply, n + 1}
end
```

This works until any other message arrives, such as a `:DOWN` from a monitor, a stray reply, or a message from a library. Defining `handle_info/2` removed the logging default, so the server crashes with `FunctionClauseError` (verified). Fix: end with a catch-all clause that logs and keeps state:

```
def handle_info(msg, state) do
  Logger.warning("unexpected message: #{inspect(msg)}")
  {:noreply, state}
end
```

4.5 Calling your own client API from a callback

```
def handle_cast(:refresh, state) do
  current = get(:rates) # get/1 is GenServer.call(__MODULE__, ...)
  {:noreply, update(state, current)}
end
```

It reads naturally, since `get/1` is right there in the module, but the callback is already inside the server. The call exits with `:calling_self` and the server crashes (verified). Fix: inside callbacks, work on `state` directly (`Map.get(state, :rates)`); client functions are for other processes.

4.6 A periodic job that queues up

```
def handle_info(:sync, state) do
  Process.send_after(self(), :sync, 1_000)
  {:noreply, do_sync(state)} # sometimes takes 3 s
end
```

The next `:sync` is scheduled before the work runs. When the work takes longer than the interval, messages arrive faster than they are handled and the mailbox grows without bound. Fix: schedule the next run *after* the work, so the gap is measured from the end of the last run.

4.7 Returning the whole state to filter elsewhere

```
def active_sessions do
  GenServer.call(__MODULE__, :get_state)
  |> Map.values()
  |> Enum.filter(& &1.active)
end
```

Each call copies the entire state, possibly hundreds of megabytes, into the caller just to keep a few entries. The process anti-patterns guide calls this *sending unnecessary data*. Fix: filter in the server (`handle_call(:active_sessions, ...)`) and return only what was asked for. If the data is read far more than written, keep it in ETS so readers don't go through the process at all.

5. Review checklist and sources

- ☐ The GenServer models state, concurrency or failure; no GenServers for code organisation
- ☐ All interaction goes through client functions in the server's module
- ☐ `init/1` returns quickly; slow setup is in `handle_continue/2`
- ☐ Names are configurable; nothing assumes a single instance

- Every custom `handle_info/2` ends with a catch-all clause
- Callbacks never call the module's own client API
- Periodic work reschedules after it finishes
- Calls return only the data the caller needs

Sources

- GenServer: <https://hexdocs.pm/elixir/GenServer.html>
- Process-related anti-patterns: <https://hexdocs.pm/elixir/process-anti-patterns.html>
- Erlang book, chapter 5 (gen_server and gen_statem): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>